

实验作业二 ConvLSTM 的算法应用与改进

(对应教材第 4 章《循环神经网络》·时空序列预测·使用 TRAE IDE 完成)

一、实验目的

- 理解 ConvLSTM“把 LSTM 的矩阵乘法替换为卷积”的核心思想，掌握时空序列预测任务的建模流程；
- 掌握 ConvLSTM 细胞的参数量计算公式，能对给定配置逐项手算并与代码核对；
- 在自合成的弹跳小球序列上完成基线 ConvLSTM 的“看过去几帧、预测下一帧”任务；
- 通过控制变量实验理解层数、隐藏通道、卷积核、输入帧数与网络结构对预测质量的影响；
- 熟练使用 TRAE IDE 的 AI 对话、代码解释与错误修复能力，规范记录人机协作过程。

二、实验环境

表 1 软件环境清单

类别	名称 / 版本要求	备注
操作系统	Windows 10/11 或 macOS	—
Python	3.10 及以上	—
PyTorch	2.0 及以上 (CPU 可跑)	无内置 ConvLSTM，需自写细胞
NumPy / Matplotlib	最新版	数据合成与结果可视化
数据集	弹跳小球序列 (脚本自动生成)	无需联网下载
IDE	TRAE IDE (含 AI Chat / Builder)	全程人机协作

环境搭建命令

```
python -m venv venv_lab02
venv_lab02\Scripts\activate          # Windows 激活虚拟环境
pip install torch numpy matplotlib
python -c "import torch; print(torch.__version__)"
```

三、TRAE IDE 使用要点

表 2 TRAE 核心功能与本实验的使用原则

功能	本实验中的用法	使用原则
AI Chat	解释 ConvLSTM 细胞代码、对比卷积版与	先自己推导，再用 AI 验证

	矩阵版公式	
Builder	生成数据合成与训练框架代码	生成后逐行读懂再运行
内联补全	补全可视化代码（画帧网格图等）	只接受理解过的补全
智能修复	定位形状不匹配（B,T,C,H,W）类报错	弄清报错原因，不盲目点修复

四、原理回顾

4.1 从 LSTM 到 ConvLSTM

LSTM（全连接版）：	$i_t = \sigma(W_{xi} \cdot x_t + W_{hi} \cdot h_{t-1} + b_i)$
ConvLSTM（卷积版）：	$i_t = \sigma(W_{xi} * X_t + W_{hi} * H_{t-1} + b_i)$
其中 *	表示二维卷积； f_t 、 $\tilde{c}_t$ 、 o_t 同理替换。
细胞状态更新不变：	$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{c}_t$
隐藏状态输出：	$H_t = o_t \odot \tanh(C_t)$

关键区别：全连接 LSTM 把图像展平成一维向量，空间关系被打散，且参数量随分辨率平方增长；ConvLSTM 保留二维特征图结构，用“局部感受野 + 参数共享”建模时空数据，这正是气象降水临近预报（雷达回波外推）等任务的标准工具。

4.2 参数量公式与基线配置

ConvLSTM 细胞（合并实现，4 个门共用一个卷积）：
每个门： $K \times K \times C_{in} \times C_h + K \times K \times C_h \times C_h + C_h$
细胞总计 = 4 × 上式
输出卷积： $K \times K \times C_h \times C_{out} + C_{out}$

表 3 基线 ConvLSTM 配置（必选手算验证）

项目	配置	参数量（手算）
任务设定	看前 4 帧（32×32），预测第 5 帧	—
ConvLSTM 细胞	$C_{in}=1, C_h=32, K=3, 1$ 层	_____
输出卷积	$\text{Conv2d}(32 \rightarrow 1, 3 \times 3, \text{padding}=1)$	_____
损失 / 优化器	MSE / Adam, lr=0.001	—
批大小 / 轮数	64 / 5 epoch	—
随机种子	42	—

表 4 六个设计维度与预期影响

维度	改动方式	预期影响
加深层数	1 层 → 2 层 ConvLSTM	时空特征更抽象，训练更慢
隐藏通道	$C_h 32 \rightarrow 64$	容量增大，精度略升，参数量约 4 倍
卷积核	$K=3 \rightarrow K=5$	感受野更大，捕捉更大范围运动
输入帧数	看 4 帧 → 看 8 帧	运动信息更充分，预测更稳
结构对照	ConvLSTM → 全连接 LSTM（展平）	体会卷积结构在时空任务中的优势
损失函数	MSE → L1（MAE）	对个别误差大的帧更鲁棒
最优组合	综合有效改进	MSE 明显下降

五、实验任务

在脚本自动生成的弹跳小球序列（2000 条训练 + 200 条测试，每条 10 帧 32×32）上：

（1）任务一：实现基线 ConvLSTM，用前 4 帧预测第 5 帧，记录测试 MSE、MAE 与训练耗时；（2）任务二：手算表 3 中细胞与输出卷积的参数数量并用代码核对；（3）任务三：完成表 4 前 6 组对照实验，每组只改一个变量；（4）任务四：将有效改进组合成最优组合，固定种子复测 3 次取平均；（5）任务五（必做）：绘制“输入帧 | 真实下一帧 | 预测帧”对比图，直观评价预测质量。

六、实验步骤

步骤 1 用 TRAE Builder 生成实验框架

请在当前目录创建 `convlstm_bounce.py`，实现弹跳小球时空预测实验：

1. 数据合成：NumPy 生成 `n_seq=2000` 条序列（测试 200 条），每条 10 帧 32×32，小球半径 2，随机初速度 0.8~1.6 像素/帧，边界反弹，随机种子 42；
2. 定义 `ConvLSTMCell`：一个 `Conv2d(in_ch+hid_ch, 4*hid_ch, k, padding=k//2)`，前向中 `cat([x, h])` 后过卷积，`chunk` 成 4 份分别过 `Sigmoid/Tanh`，按 `c_t = f*c + i*g`、`h_t = o*tanh(c)` 更新；
3. 定义 `ConvLSTM` 模型：沿时间循环调用细胞，最后用 `Conv2d(32→1, 3×3, padding=1)` 从最后隐藏状态预测下一帧；层数、hidden、kernel 可配置；
4. 训练：MSE 损失 + `Adam(0.001)`，每 epoch 记录训练损失与测试 MSE/MAE；
5. 结束后保存损失曲线 `result_baseline.png`，以及“输入 4 帧 | 真实帧 | 预测帧”九宫格对比图 `result_pred.png`（中文字体用 `Microsoft YaHei`）。

提示：数据张量形状为 (B, T, C, H, W)；ConvLSTM 沿时间循环时逐步“喂”每一帧 `x[:, t]`。

步骤 2 手算验证（必做）

用 4.1 节公式计算基线细胞参数量（ $K=3$ 、 $C_{in}=1$ 、 $C_h=32$ ）与输出卷积参数量，写出代入过程填入报告表 3，并用 `sum(p.numel() for p in model.parameters())` 核对总参数量。

步骤 3 运行基线并保存曲线

运行基线（CPU 上约 10–20 分钟），把测试 MSE、MAE 与耗时填入表 5，曲线保存为 `result_baseline.png`。

步骤 4 逐组完成对照实验

按表 5 逐组实验：每次只改动一个变量，其余与基线一致；每组保存曲线图（result_exp1.png ~ result_exp6.png）。实验 5（全连接 LSTM 对照）需另写 FlattenLSTM 模型类：展平 4 帧（4×1024 维）输入 nn.LSTM(1024, 256)，输出 Linear(256→1024) 重塑成 32×32。

步骤 5 最优组合与可视化

把有效改进合成最优组合，固定种子运行 3 次取平均；对测试集 3 个不同样本绘制“输入 | 真实 | 预测”对比图 result_pred.png，标注每个样本的 MSE。

表 5 对照实验设计与结果记录表

组号	实验名称	改动内容（相对基线）	测试 MSE	测试 MAE	耗时 (s)	观察结论
0	基线 Baseline	1 层, C_h=32, K=3, 看 4 帧, MSE	_____	_____	_____	_____
1	加深层数	2 层 ConvLSTM	_____	_____	_____	_____
2	隐藏通道	C_h 32 → 64	_____	_____	_____	_____
3	卷积核	K=3 → K=5	_____	_____	_____	_____
4	输入帧数	看 4 帧 → 看 8 帧	_____	_____	_____	_____
5	结构对照	全连接 LSTM（展平）	_____	_____	_____	_____
6	损失函数	MSE → L1	_____	_____	_____	_____
7	最优组合	综合有效改进	_____	_____	_____	_____

七、参考代码（完整框架）

```
import time
import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn as nn

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei"]
plt.rcParams["axes.unicode_minus"] = False
torch.manual_seed(42); np.random.seed(42)

# ----- 数据合成：弹跳小球 -----
def make_sequences(n_seq=2000, T=10, size=32, r=2, seed=42):
    rng = np.random.default_rng(seed)
    yy, xx = np.mgrid[0:size, 0:size]
    seqs = np.zeros((n_seq, T, size, size), np.float32)
    for s in range(n_seq):
        x, y = rng.uniform(4, size - 5, 2)
        vx, vy = rng.choice([-1, 1], 2) * rng.uniform(0.8, 1.6, 2)
        for t in range(T):
            x, y = x + vx, y + vy
            if x < r or x > size - r: vx = -vx
            if y < r or y > size - r: vy = -vy
            seqs[s, t] = ((xx - x) ** 2 + (yy - y) ** 2 <= r * r)
    return seqs[..., None] # (N, T, 32, 32, 1)
```

```

data = make_sequences()
K_IN = 4                                # 看前 4 帧
train_x = torch.tensor(data[:2000, :K_IN]).permute(0, 1, 4, 2, 3)
train_y = torch.tensor(data[:2000, K_IN]).squeeze(-1)
test_x = torch.tensor(data[1800:, :K_IN]).permute(0, 1, 4, 2, 3)
test_y = torch.tensor(data[1800:, K_IN]).squeeze(-1)

# ----- ConvLSTM -----
class ConvLSTMCell(nn.Module):
    def __init__(self, in_ch, hid_ch, k=3):
        super().__init__()
        self.conv = nn.Conv2d(in_ch + hid_ch, 4 * hid_ch,
                                k, padding=k // 2)
        self.hid = hid_ch

    def forward(self, x, state):
        h, c = state
        z = self.conv(torch.cat([x, h], dim=1))
        i, f, g, o = z.chunk(4, dim=1)
        i, f, o = torch.sigmoid(i), torch.sigmoid(f), torch.sigmoid(o)
        c_new = f * c + i * torch.tanh(g)
        h_new = o * torch.tanh(c_new)
        return h_new, c_new

class ConvLSTM(nn.Module):
    def __init__(self, in_ch=1, hid=32, k=3, layers=1):
        super().__init__()
        chs = [in_ch] + [hid] * layers
        self.cells = nn.ModuleList(
            [ConvLSTMCell(chs[i], chs[i + 1], k)
             for i in range(layers)])
        self.out = nn.Conv2d(hid, 1, 3, padding=1)

    def forward(self, x):
        # x: (B, T, C, H, W)
        b, _, _, hsize, wsize = x.shape
        states = [(torch.zeros(b, c.hid, hsize, wsize),
                    torch.zeros(b, c.hid, hsize, wsize))
                   for c in self.cells]
        for t in range(x.shape[1]): # 沿时间循环
            xt = x[:, t]
            for j, cell in enumerate(self.cells):
                states[j] = cell(xt, states[j])
            xt = states[j][0]
        return self.out(states[-1][0]).squeeze(1)

# ----- 训练与测试 -----
def run(model, loss_fn=None, lr=0.001, epochs=5, bs=64):
    loss_fn = loss_fn or nn.MSELoss()
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    n = train_x.shape[0]
    t0 = time.time()
    for ep in range(epochs):

```

```
model.train()
perm = torch.randperm(n)
for i in range(0, n, bs):
    idx = perm[i:i + bs]
    opt.zero_grad()
    loss = loss_fn(model(train_x[idx]), train_y[idx])
    loss.backward()
    opt.step()
model.eval()
with torch.no_grad():
    pred = model(test_x)
print("epoch %d  train_loss %.5f  test_MSE %.5f  test_MAE %.5f"
      % (ep + 1, loss.item(),
         (pred - test_y).pow(2).mean().item(),
         (pred - test_y).abs().mean().item()))
return time.time() - t0, pred

if __name__ == "__main__":
    model = ConvLSTM()
    print("总参数量: ", sum(p.numel() for p in model.parameters()))
    dt, pred = run(model)
    print("耗时 %.1fs" % dt)
```

注意：实验 5 的全连接 LSTM 对照需另写 FlattenLSTM 类（展平 4 帧 → nn.LSTM(1024, 256) → Linear(256→1024) → 重塑 32×32）；实验 6 的 L1 损失即 nn.L1Loss()。

八、实验报告要求

报告以 Word / PDF 提交，命名“学号_姓名_实验作业二.docx”。请直接在配套的《实验作业二_ConvLSTM 的算法应用与改进_实验报告样板.docx》中填写：结构为实验概述、基线实验（含参数量手算）、对照实验（表 5 全量数据 + 逐组分析）、AI 协作记录、结果分析（含预测帧对比图讨论）、思考题作答。

表 6 评分标准（百分制）

项目	分值	要求
基线实现与手算	20	基线可运行；细胞与输出卷积参数量手算正确
对照实验	30	6 组齐全、控制变量严格、数据真实可复现
结果分析	20	逐组有归因分析；预测帧对比讨论有理有据
AI 协作记录	15	3 处以上 TRAE 使用记录，含提示词与采纳说明
报告规范	10	表格图齐全、编号规范、语言通顺
创新加分	5	额外维度（如多步预测、球数增加、边界噪声）

九、常见问题

Q1：为什么要“自造数据”而不是用真实数据集？

弹跳小球序列物理规律清晰、本地秒级生成，便于把误差归因到模型本身；ConvLSTM 的经典应用（雷达回波外推）数据量大且需专业渠道，不适合课堂。

Q2: CPU 训练一组要多久？

基线 5 个 epoch 约 10–20 分钟，全套 8 组约 2 小时。时间紧张可先用 800 条训练序列、3 个 epoch 跑通流程，再跑全量。

Q3: 报错 expected 5D input?

ConvLSTM 输入形状必须是 (B, T, C, H, W)。数据生成时是 (N, T, H, W, 1)，参考代码用 `permute(0, 1, 4, 2, 3)` 换轴——这是本实验最常见的坑。

Q4: 为什么 PyTorch 没有内置 ConvLSTM?

`nn.LSTM` 面向通用序列；ConvLSTM 需要把矩阵乘换成卷积，官方未内置。亲手实现 `ConvLSTMCell` 正是本实验的核心训练——它就是教材 LSTM 六条公式的卷积版。

Q5: 预测帧发虚 / 有重影？

MSE 倾向输出均值模糊，属正常现象；增加输入帧数、加深网络或多步训练可缓解，L1 损失略微更锐利——可在报告中对比讨论。

Q6: 每次结果不一样？

固定随机种子：`torch.manual_seed(42)`、`np.random.seed(42)`，数据合成的 `rng` 也已固定；最优组合固定种子跑 3 次取平均。

十、思考题（在报告样板中作答）

(1) ConvLSTM 与普通 LSTM 的本质区别是什么？为什么“保留二维空间结构”对预测运动序列特别重要？

(2) 手算：若隐藏通道 C_h 从 32 增至 64 ($K=3$, $C_{in}=1$)，单个 ConvLSTM 细胞的参数量变为多少？增长了几倍？说明容量与成本的权衡。

(3) 实验 5 中全连接 LSTM 的参数量是多少（展平 4×1024 输入、隐藏 256）？与基线 ConvLSTM 相比呢？从参数量与空间局部性两方面解释其表现差异。

(4) 增加输入帧数（实验 4）为什么能提升预测精度？如果继续加到 16 帧可能遇到什么问题？

(5) 查阅资料并回答：在气象降水临近预报（雷达回波外推）中，ConvLSTM 的输入与输出各是什么？该任务与弹跳小球相比有哪些额外难点（提示：目标可生消、多尺度、真实噪声）？

(6) 本实验的 `ConvLSTMCell` 没有现成库可调用。回顾你与 TRAE 协作实现它的过程：AI 在哪些环节最有价值？哪些地方必须自己读懂公式才能改对？